

Roteiro Guiado de Refatoração

Princípios de projeto e boas práticas de orientação a objetos

Objetivo: melhorar o projeto aplicando princípios de projeto e boas práticas de orientação a objetos. Faça as alterações na ordem apresentada. Após cada etapa, execute novamente o projeto e verifique se ele continua funcionando.

Entendendo a atividade

Este projeto representa uma pequena loja acadêmica, responsável por criar e finalizar pedidos de clientes.

Classes de domínio

- **Cliente**, que possui nome e endereço.
- **Endereco**, que possui o logradouro e a cidade.
- **Cidade**, que armazena o nome da cidade.
- **Pedido**, que possui um cliente e uma lista de itens.
- **ItemPedido**, que representa um produto, seu preço e quantidade.

Formas de pagamento

Também existem diferentes formas de pagamento — **PagamentoCartao**, **PagamentoPix** e **PagamentoBoleto**. Essas classes implementam a interface **IPagamento**, que atualmente define operações de pagamento, parcelamento e geração de boleto. Algumas formas de pagamento, porém, não utilizam todas essas operações.

PedidoService e entrega

A classe **PedidoService** concentra boa parte do funcionamento do sistema. Ela é responsável por calcular o valor do pedido, aplicar descontos, descobrir a cidade do cliente, salvar informações, selecionar a forma de pagamento, gerar um resumo e finalizar o pedido.

O projeto também possui as classes **Entrega** e **EntregaRetiradaLoja**, utilizadas para representar diferentes comportamentos relacionados à entrega.

O código funciona, mas foi propositalmente desenvolvido com algumas decisões de projeto que podem ser melhoradas. Sua missão será identificar esses pontos e realizar pequenas refatorações, mantendo o comportamento da aplicação.

ETAPA 1 / 7 Organizando o PedidoService

Contexto

Abra a classe **PedidoService** e observe o método **finalizarPedido**. Atualmente ele possui várias responsabilidades diferentes: calcula o pedido, salva informações em arquivo, gera um resumo, realiza o pagamento e exibe uma mensagem para o cliente.

Sua primeira tarefa será reduzir essas responsabilidades.

O que fazer

1. Crie uma nova classe responsável exclusivamente por salvar os pedidos em arquivo. Sugestão de nome: **PedidoRepository**.
2. Crie nessa classe um método responsável por receber os dados necessários e realizar o salvamento.
3. Depois disso, altere o **PedidoService** para utilizar essa nova classe.
4. Ao terminar, execute novamente o projeto.

ETAPA 2 / 7 Melhorando a interface de pagamento

Contexto

Abra a interface **IPagamento** e observe os métodos existentes: **pagar**, **parcelar** e **gerarBoleto**.

Agora analise as classes **PagamentoCartao**, **PagamentoPix** e **PagamentoBoleto**. Perceba que nem todas as formas de pagamento utilizam todos os métodos disponíveis na interface.

O que fazer

1. Divida **IPagamento** em interfaces menores. Por exemplo, uma interface responsável por pagamento e outras específicas para funcionalidades como parcelamento ou geração de boleto.
2. Depois disso, faça cada classe implementar somente as interfaces que realmente fazem sentido para ela.
3. Ao finalizar, execute novamente o projeto.

ETAPA 3 / 7 Removendo uma herança inadequada

Contexto

Observe a declaração da classe **PedidoService**. Atualmente ela herda de **PagamentoCartao**.

Pergunte-se

Um **PedidoService** é realmente um tipo de **PagamentoCartao**?

O que fazer

1. Remova essa herança.
2. **PedidoService** deve utilizar um objeto responsável pelo pagamento, e não se transformar em uma forma de pagamento.
3. Utilize composição para representar essa relação.
4. Execute novamente o projeto.

ETAPA 4 / 7 Reduzindo o conhecimento entre objetos

Contexto

Observe o método responsável por obter a cidade de entrega. Atualmente o código precisa percorrer vários objetos:

pedido → **cliente** → **endereço** → **cidade** → **nome**

Isso faz com que **PedidoService** conheça muitos detalhes internos de outras classes.

O que fazer

1. Crie métodos nas próprias classes para esconder parte desse caminho. Por exemplo, **Cliente** pode fornecer sua cidade de entrega sem que **PedidoService** precise conhecer toda a estrutura interna do endereço.
2. Depois da alteração, o **PedidoService** deverá utilizar uma chamada mais simples para descobrir a cidade.
3. Execute novamente o projeto.

ETAPA 5 / 7 Melhorando o cálculo de desconto

Contexto

Observe o método **calcularTotal**. Atualmente existem vários if/else verificando o tipo do cliente: **ALUNO**, **PROFESSOR** e **FUNCIONARIO**.

Imagine agora que futuramente sejam criados novos tipos de clientes. Cada novo tipo exigiria alteração dentro do **PedidoService**.

O que fazer

1. Retire a responsabilidade de decidir o desconto de dentro do **PedidoService**.
2. Crie uma abstração responsável pelo cálculo do desconto.
3. Depois crie implementações diferentes para os tipos de desconto existentes.
4. **PedidoService** deverá apenas utilizar o cálculo recebido, sem precisar descobrir qual tipo de cliente está sendo processado.
5. Execute novamente o projeto.

ETAPA 6 / 7 Removendo os if/else de pagamento

Contexto

Volte ao método **finalizarPedido**. Observe que existem verificações como **CARTAO**, **PIX** e **BOLETO**.

Novamente, pense no que aconteceria se uma nova forma de pagamento fosse adicionada.

O que fazer

1. Faça com que o **PedidoService** receba uma forma de pagamento por meio de uma interface.
2. O serviço não deverá mais decidir qual classe de pagamento criar.
3. Quem utilizar o **PedidoService** deverá fornecer a implementação correta.
4. Depois disso, remova os if/else responsáveis por escolher a forma de pagamento.
5. Execute novamente o projeto.

ETAPA 7 / 7 Analisando EntregaRetiradaLoja

Contexto

Observe as classes **Entrega** e **EntregaRetiradaLoja**. A classe **Entrega** permite calcular um frete normalmente. Entretanto, **EntregaRetiradaLoja** pode lançar uma exceção dependendo do valor do pedido.

O que fazer

1. Analise se **EntregaRetiradaLoja** realmente pode substituir **Entrega** em qualquer situação.
2. Refatore essa estrutura para que as diferentes formas de entrega possuam regras claras e previsíveis, sem quebrar as expectativas de quem utiliza a abstração.

3. Execute novamente o projeto.

Finalizando a atividade

Depois de concluir todas as etapas:

- Execute o programa completo.
- Verifique se o pedido continua sendo calculado corretamente.
- Verifique se o pagamento continua funcionando.
- Verifique se as informações do pedido continuam sendo apresentadas.
- Compare a estrutura original com a estrutura refatorada.



Reflexão final

Quais classes ficaram com responsabilidades mais claras depois da refatoração?



Reflexão final

Se fosse necessário adicionar uma nova forma de pagamento ou um novo tipo de desconto, quantas classes precisariam ser alteradas agora?

Roteiro baseado em princípios de projeto e boas práticas de orientação a objetos.